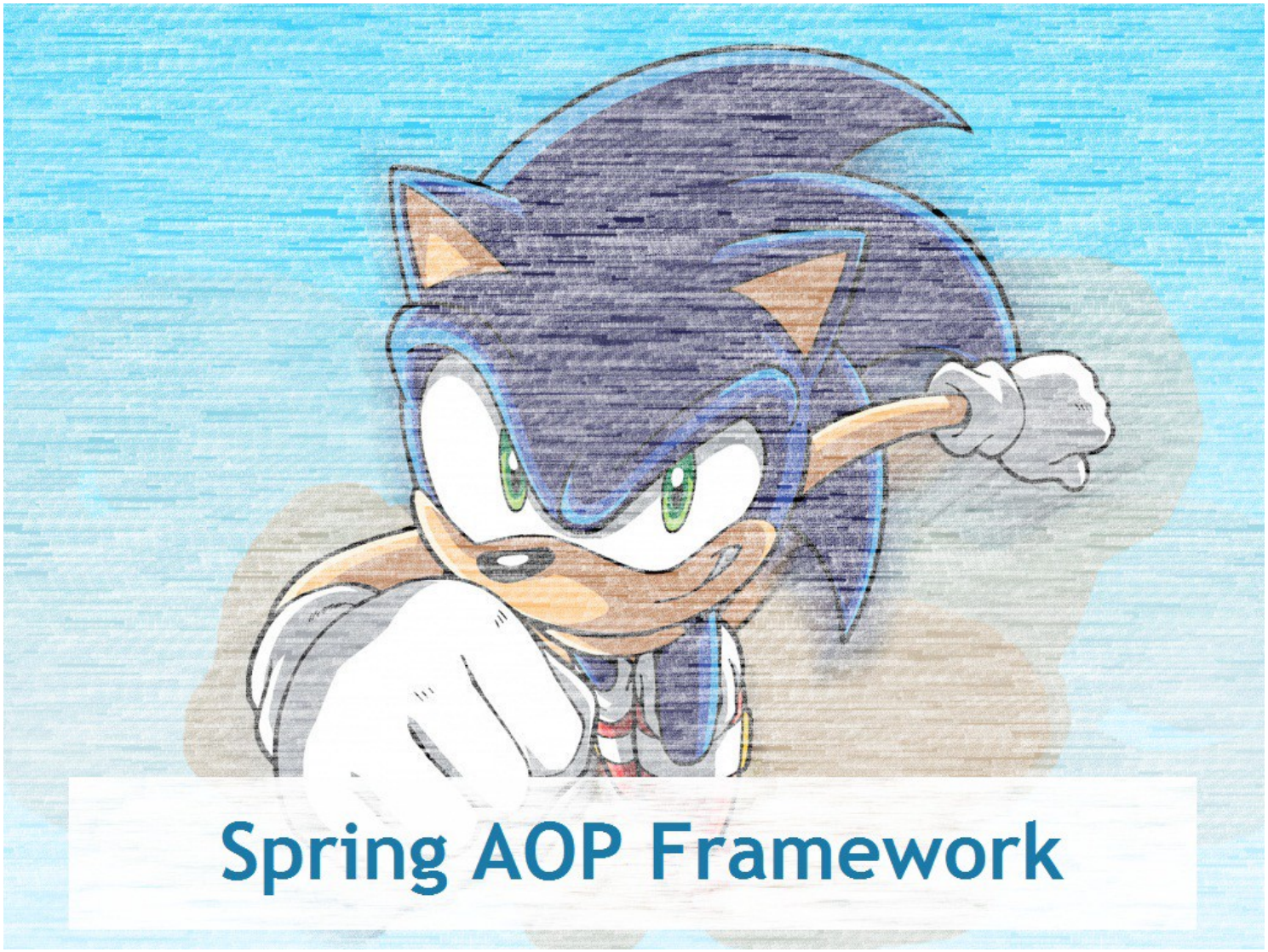
# Spring: A Head Start 🔥 — Spring AOP Framework (Part 4)

The separation of cross-cutting concerns from the business logic.

Omar Elgabry Follow
Sep 12, 2017 · 3 min read




Spring: A Head Start 🔥 — Spring AOP Framework

The AOP (Aspect Oriented Programming) in Spring was discussed earlier in Spring: A Head Start 🔥 — Introduction (Part 1).

Aspect Oriented Programming (AOP) basically helps you to decouple *cross-cutting concerns* from the objects that they affect.

## AOP Terminologies

AOP has some terminologies and concepts that one should be familiar with.

- **Aspect**: A module (class) where you define a cross-cutting concern code.
- **Joint Point**: It's a point in the application where where an aspect can be plugged in.

> 💡*It could be a method being called, an exception being thrown, or a field being modified.*

- **Advice**: The actual code (i.e. method) of the aspect to be executed at different, and any join points. It has 5 different types.
- **Pointcut:** It defines at what join points, the associated advice(s) should be executed.

### Types of Advice

- **before** →Run advice before the method execution.
- **after** → Run advice after the method execution, regardless of its outcome.
- **after-returning** →Run advice after the method execution, only if the method completes successfully.
- **after-throwing** → Run advice *only* after the method throws an exception.
- **around** →Run advice before and after the advised method is invoked.

## Implementation

Spring supports the XML syntax, or syntax based on "AspectJ", which uses Java code to implement custom aspects.

To get our hands dirty, and understand what we can do with AOP Framework, let's implement a Logging aspect using XML syntax.

### The User Class

This is just a simple class so we can fire the advice code of an aspect at different points; at different methods in the User class.

```

public class User {
  private Integer id;
  private String name;

  public void setId(Integer id) {
    this.id = id;
  }

  public Integer getId() {
    System.out.println("Id: " + id);
    return id;
  }

  public void setName(String name) {
    this.name = name;
  }

  public String getName() {
    System.out.println("Name: " + name );
    return name;
  }

  public void printThrowException(){
    System.out.println("Exception raised");
    throw new IllegalArgumentException();
  }
}
```

User.java hosted with ♡ by GitHub view raw

## The Aspect — Logging

This is a simple aspect module which defines the methods to be called at different points.

```

public class Logging {
  public void beforeAdvice(){
    System.out.println("Before Advice");
  }

```

```
  public void afterAdvice(){
    System.out.println("After Advice");
  }

  public void afterReturningAdvice(Object retVal){
    System.out.println("After Advice Executed Successfully ... Returning: " + retVal.toString() );
  }

  public void AfterThrowingAdvice(IllegalArgumentException ex){
    System.out.println("There has been an exception when executing the advice: " + ex.toString());
  }
}
```

Logging.java hosted with ♡ by GitHub view raw

> *`retVal` is the returned value from the joint points (methods) where the aspect is applied. In our case, If the joint point is on `getName()` method of User class, then `retVal` will be assigned to the user's name.*

## The Configurations

Basically, we need to define the aspect, and reference the aspect bean. So, obviously, the aspect class has to be defined as a bean.

Then, for each aspect, we define the Pointcuts, which determine the join points (i.e. methods) where the advice(s) will be executed.

The following example defines a Pointcut named 'UserAllMethod' which matches all the methods in User class.

> *If you want to apply to a specific method instead of the whole class methods, then use `execution(* User.getName(..))` .*

Finally, define the advice; the actual method code that will run at (before or after) the join point(s) we've defined.

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
        xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xmlns:context="http://www.springframework.org/schema/context"
        xmlns:aop="http://www.springframework.org/schema/aop"
```

```xml
        xsi:schemaLocation="
        http://www.springframework.org/schema/beans
        http://www.springframework.org/schema/beans/spring-beans-3.0.xsd
        http://www.springframework.org/schema/context
        http://www.springframework.org/schema/context/spring-context-3.0.xsd
        http://www.springframework.org/schema/aop
        http://www.springframework.org/schema/aop/spring-aop-3.0.xsd ">

   <aop:config>
     <!-- Aspect -->
     <aop:aspect id="loggingAspect" ref="logging">

       <!-- Pointcut -->
       <aop:pointcut id="UserAllMethods" expression="execution(* User.*(..))"/>

       <!-- Advice(s) -->
       <aop:before pointcut-ref="UserAllMethods" method="beforeAdvice"/>
       <aop:after  pointcut-ref="UserAllMethods" method="afterAdvice"/>
       <aop:after-returning pointcut-ref="UserAllMethods" returning="retVal" method="afterReturning
       <aop:after-throwing  pointcut-ref="UserAllMethods" throwing="ex" method="AfterThrowingAdvice
     </aop:aspect>
   </aop:config>

   <!-- The user bean -->
   <bean id="user" class="User">
           <property name="name" value="Alex" />
           <property name="id" value="125"/>
   </bean>

   <!-- Definition for logging aspect -->
   <bean id="logging" class="Logging"/>
 </beans>
```

beans.xml hosted with ♡ by GitHub view raw

> ⚠ *Make sure you have added the required JAR files or define the dependencies in "pom.xml" file in case of using Maven.*

```xml
<dependency>
  <groupId>org.springframework</groupId>
  <artifactId>spring-aop</artifactId>
  <version>${springframework.version}</version>
</dependency>
```

```
<dependency>
  <groupId>org.aspectj</groupId>
  <artifactId>aspectjweaver</artifactId>
  <version>1.8.10</version>
</dependency>
```

## Testing Our Aspect

Now, it's time to see the outcome.

```
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

public class Test {
  public static void main(String[] args) {
    ApplicationContext context = new ClassPathXmlApplicationContext("beans.xml");

    User user = (User) context.getBean("user");
    user.getName();
    user.printThrowException();
  }
}
```

Test.java hosted with ♡ by GitHub view raw

Thank you for reading! If you enjoyed it, please clap 👏 for it.

Java Spring Framework Programming Software Development Other

About Help Legal

Get the Medium app

Download on the App Store GET IT ON Google Play

App Store Google Play